

Project 1.4: Twitter Sentiment Analysis (Group 28)

Lawrence May
M. Sc. Computer Science
Technische Universität Berlin
Berlin, Germany
l.may@campus.tu-berlin.de

Nick Samuel Fath
M.Sc. ISM
Technische Universität Berlin
Berlin, Germany
n.fath@campus.tu-berlin.de

Berk Bubus
M. Sc. Computer Science
Technische Universität Berlin
Berlin, Germany
bubus@campus.tu-berlin.de

ABSTRACT

Twitter, since 2023 known as X, is a social network where users comment (*tweet*) blog-style or reply to other comments. The comments themselves are limited in length, for a long time at most 140 characters. The Twitter Sentiment data set contains a collection of tweets with sentiment classification. Our task was to use the data set in the context of natural language processing to train neural network models for sentiment inference.

We compared a Naïve-Bayes classifier with a feed-forward neural network, testing various pre-processing steps, highlighting the importance of parameter tuning.

Further, we've trained a LSTM recurrent neural network using BERT text embeddings as inputs. As a rather advanced language model, this resulted in a significant improvement to standard embedding techniques.

1. The data set

The Twitter sentiment data set contains 27,480 tweets. It contains seven columns: *textID*, *text*, *selected_text*, *sentiment*, *Time of Tweet*, *Age of User*, *Country*, *Population*, *Land Area* and *[population] Density*. For this project, we will only focus on the *text* and *sentiment* columns.

The tweets are classified as having either positive, neutral or negative sentiment. Regarding the distribution, 11,117 of them are neutral, 8,582 positive and 7,781 negative. It therefore has a slight class imbalance, in percentage terms 41% are neutral, 31% positive, and 28% negative. Splitting the tweets by whitespace into tokens, there are 52,270 unique tokens in the data set. The tweets are roughly equal in length regarding the word count, with negative tweets being slightly longer than positive tweets and neutral tweets being the shortest.

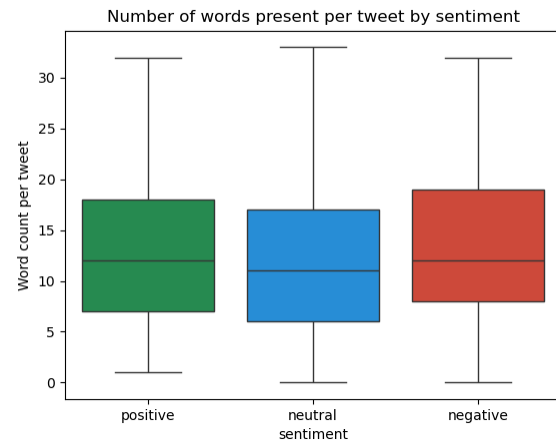


Figure 1: Box plot of the distribution of words present per tweet by sentiment

The tweets vary in length up to 140 characters due to the character length limitations of tweets at the time of data collection. Some characters are garbled due to encoding issues, possibly during data scraping. Swear words in the data set are censored, their characters being replaced with asterisks. There are no emojis present, only a few regular smileys like ;-). A few hashtags (*#hooray*), mentions (*@twitteruser123*) and URLs are present in the data,

The Fleisch Reading Ease scores of the tweets are, on average, 82.76 for positive, 83.64 for neutral and 84.39 for negative tweets, indicating that there is not a large gap regarding the difficulty of reading either sentiment's tweets, all of them being easy to read on average.

The most common words in positive tweets contain *love*, *good*, and *thank*. The most common words in neutral tweets contain *lol* and *going*, with negative tweets containing *work*, *sad*, and *miss*. All the sentiments heavily feature the word *day*, both neutral and negative tweets often feature *work* or *now*, indicating a significant overlap of frequent words between the categories, which adds to the difficulty of inferring sentiment from text. A few strong category-specific words indicate an opportunity for differentiation.



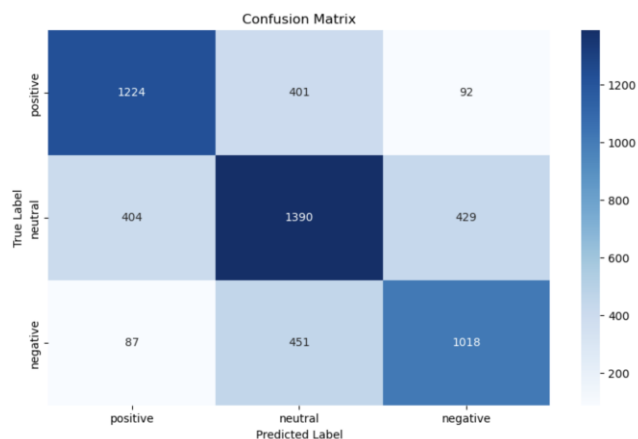


Figure 3: Confusion Matrix of Naïve Bayes model

3.3.4 Model error analysis.

We didn't see anything too noteworthy here, other than confirming that the model somewhat struggles to correctly classify neutral tweets. However, for some of these tweets we thought we would struggle to correctly classify them/ would have wrongly classified them. Given this knowledge, a combined score of 0.66 using a simple Naïve Bayes model looks like a good start.

3.4 Feed-Forward neural network model

3.4.1 Pre-processing

Again, theoretically both Lemmatisation and Stemming appeared to be valid options for word normalization for the neural network model. Unfortunately, we lacked computational resources to perform a comprehensive grid search as with the Naïve Bayes model. We therefore manually tried a couple of different pre-processing steps, again comparing lemmatisation and stemming, as well as TF-IDF and BoW. TF-IDF appeared to work slightly better here, this could also have been due to chance however as we were not able to cross-validate this insight. However, we still decided to go with this approach here, with the theoretical justification that a neural network might better be able to utilise the more complex structure of TF-IDF vectorisation compared to a simpler Naive-Bayes classifier. The notable exception here is the inclusion of stop words, which almost always led to a superior model. Given this similarity in performance, we again decided to go with the simpler approach of unigram bag-of-words vectorization combined with stemming.

3.4.2 Model parameters

As with pre-processing, we lacked the computational resources to perform a comprehensive grid search to extensively tune the neural network's hyperparameters. We therefore again manually worked through different combinations of hyperparameters that

we thought would have the biggest impact on model performance, such as the number of neurons and hidden layers, and the learning rate. We also used early stopping, as this tends to be a rare "free lunch" - both reducing training time and preventing overfitting.

This manual tuning approach indicated a slighter deeper network, we ended up using 2 hidden layers with 128 and 64 neurons, respectively, turned out to perform better than a shallower one. A slower learning rate of 0.0001 also added some improvements. The remaining hyperparameters were left mostly on the default settings due to our computational constraints. Given that we could not cross-validate any model tuning, there always was a risk of overfitting present that lead us to prefer using the standard parameters.

3.4.3 Model performance

Our Neural Network offers a slight improvement over the Naïve Bayes one, with a combined f1-score of 0.68. This does partially however also reflect the fact that it is largely untuned, with out-of-the box hyperparameters and pre-processing options, compared with the highly tuned and customized pipeline we were able to implement for the Naïve bayes model due to its much faster training time. Given more computational resources for training, there likely is significant room for improvement for this model's performance.

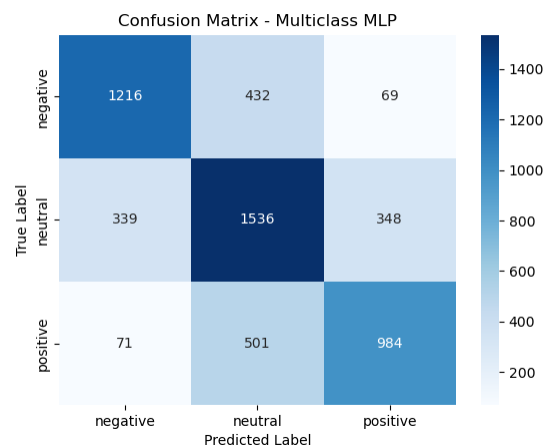


Figure 4: Confusion Matrix of Feed-Forward Neural Network

3.4.4 Model error analysis

Again, nothing too noteworthy here, other than confirming that the model somewhat struggles to correctly classify neutral tweets.

3.4.5 Dropping the neutral class

Given our previous observation that the model performs relatively well distinguishing between positive and negative, but struggles with the neutral sentiment class, dropping the neutral class seemed like a reasonable approach to improve overall performance. And indeed, doing so lifted our overall f1-score

from 0.68 in the multiclass setting to a much-improved 0.87.

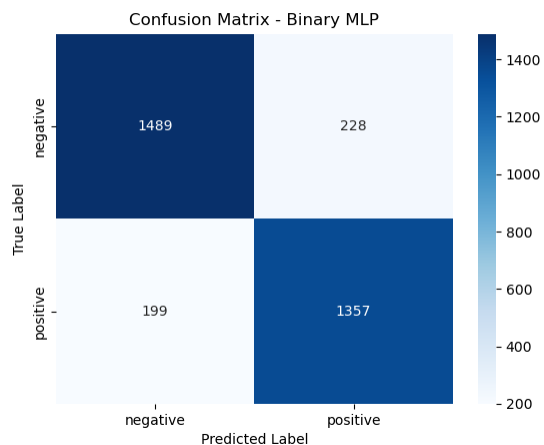


Figure 5: Confusion Matrix of Binary MLP

4. PMI-based word similarity

We've closely followed the formulas given in the lecture slides, first calculating individual word probabilities, then joint probabilities for each pair, and lastly the PMI for each word pair combination. For ten randomly chosen words, we've ended up with the following respective most-common words: *drumstick* and *fry*, *slapton* and *stupid*, *sucker* and *for*, *foa* and *real*, *holdin* and *kale*, *happiletter* and *my*, *morningmidday* and *nation*, *hudgens* and *vanessa*, *depresses* and *me*, *judy* and *same*.

5. Pre-trained embedding and RNN model

1.1 Model choice

Given computational resources required to fine-tune a BERT model, using a pre-trained model as an input embedding and training an RNN on this input embedding seemed like a good choice. After comparing the various BERT models, besides standard BERT, that are available, we decided to use the "cardiffnlp/twitter-roberta-base-emotion" pre-trained BERT model instead. This has already been fine-tuned on twitter data and (perhaps unsurprisingly) performs better than standard BERT on our data.

1.2 Training process

Training consisted of two steps: 1.) creating the embeddings for our tweet data and 2.) fitting a recurrent neural network (RNN) on these embeddings.

Step 1) was relatively straight-forward, given we simply freeze model weights here and no new parameters had to be estimated. An issue we ran into was that all the embeddings did not fit in my machine's memory, causing

the program to crash during the embedding process. The solution here was to save them to the disk using memory-mapping via the Numpy library (`np.memmap`). This technique created a large array directly on disk, allowing us to process the data in batches and write the resulting embedding vectors directly to this file.

Step 2) Fitting the RNN to the embeddings was somewhat more complex, given the relatively complex design choices we faced. We chose to use an LSTM model, which has better long-term memory than a standard RNN, likely leading to better performance for this task. The rationale for this is that, for some words, even if they are at the end of the sentence, some context at the very beginning of the sentence can be important. Therefore, for the model to correctly predict the sentiment of a tweet, having a good understanding of the entire tweet is important, which is where better long memories come in handy. Another nice property of LSTMs is that we can make them bidirectional, which allows the model to not just read tweets from start to finish, but also backwards. By looking at the sentence from both directions, the model gets a much richer understanding of the context around every word, helping it make a more accurate decision about whether the sentiment is positive, negative, or neutral.

1.3 Model parameters

Ideally, a cross-validated grid search like the model fitting process used for the Naïve Bayes model would have been used to estimate the LSTM's various hyperparameters in a structured and scientific way. However, given the computational demands of training LSTM's this would not have been feasible with the computational resources we had at our disposal. Therefore, we used a relatively standard model architecture with 2 stacked, bidirectional layers with 256 neurons each. Inputs are processed in both directions, and the output of the final layer for both directions is concatenated, giving us an output feature vector of dimension 512. We then pass this into a final, fully connected layer with three output neurons (or two in the binary case).

Furthermore, a learning rate of 0.001, batch_size of 64, cross-entropy loss and the Adam optimizer are used. All of these are relatively standard and have a high degree of robustness. To prevent overfitting, we add dropout layers after each LSTM layer with a dropout rate of 20%. We also employ early stopping, which is a great way to both prevent overfitting and speed up training time. As we will see in the next section, these relatively standard architecture and parameters (in a sentiment analysis context) lead to quite satisfactory performance, therefore we decided to use them for our final model. However, there certainly is still room to further fine-tune them, especially with more specialized computational resources.

1.4 Model performance multiclass

For the multiclass setup, we can report a combined f1-score of 0.75, which is a marked improvement on the

conventional embedding approaches we used in section 3 where we got to an f1-score of 0.66 for Naïve Bayes and 0.69 for the standard neural network.

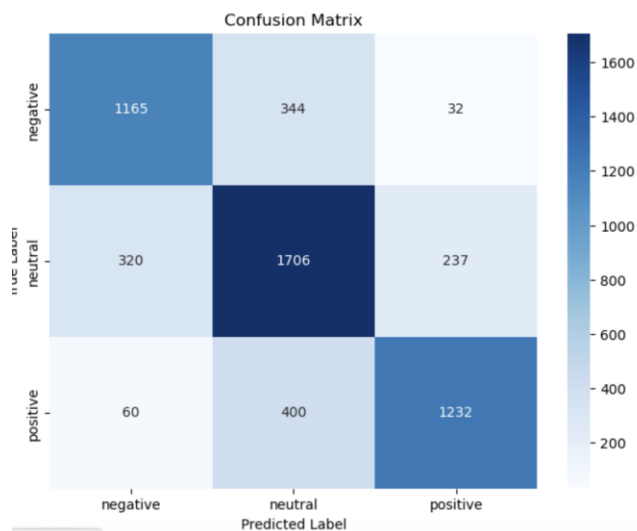


Figure 6: Confusion Matrix of Multiclass RNN

1.5 Model performance binary class

For the binary setup, we can report a combined f1-score of 0.92, which again is a significant improvement on the conventional embedding approaches we used in section 3 where we got to an f1-score of 0.87 for the feed-forward neural network.

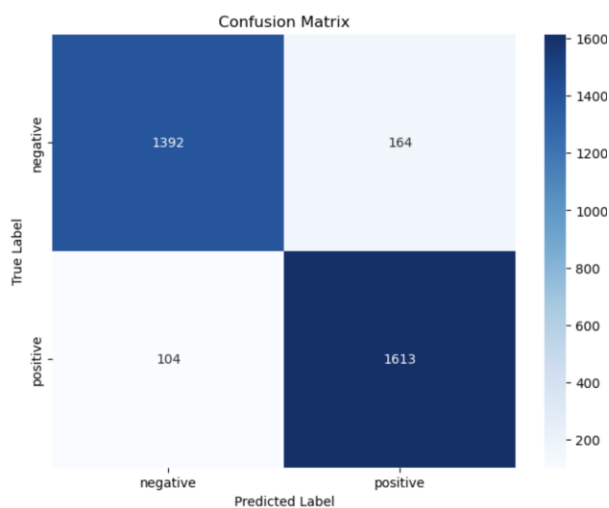


Figure 7: Confusion Matrix of Binary Multiclass RNN